

17/08/23

Page no: _____

Date: ____/____/____

* Unit III : Process Management [14/11]

▪ Process commands

- ① \$ps - ps basically means 'process status'. It displays characteristics of a process (i.e. terminal number, time required, PID no. and command name)

Options:

- a) ps -f: Used to display full listing of attributes of a process. It includes user ID, PPID (Parent PID), C (amount of CPU time consumed by process) and STIME (chronological time that has elapsed since process started). Syntax: \$ps -f
- b) -u: Shows activities of any specified user at any time. Syntax: \$ps -u username
- c) -a: It shows the processes of all users. Syntax: \$ps -a
- d) -e: Displays processes including user & system processes. Syntax: \$ps -e
- ② \$wait - Used to wait until all background processes are completed & then exits. Syntax: \$wait

③ \$sleep - It is used to execute commands after certain amount of time by sleeping for given seconds.
Syntax: \$ sleep Number
eg: \$ sleep 10 → Execute command after 10 seconds

④ \$exit - It is used to quit the shell.
Syntax: \$exit.

⑤ \$kill - Used to stop execution of particular process by sending an interrupt signal to process. Syntax: \$kill

Options:

a) \$kill 0: Kills all processes on terminal except the login shell by special argument.

b) \$kill <pid>: Kill the specified processes by pid.

Syntax: \$kill 123 456 897

c) \$kill -9 0: Kills all processes including login shell

d) \$kill -9 \$\$: Kills login shell

Program

- i) Contains set of instructions designed to complete a specific task
- ii) It is a passive entity as it resides in secondary memory
- iii) It is a static entity
- iv) It does not have any control block
- v) Exists at single place & continues to exist until it is deleted

Process

- It is an instance of an executing program
- It is an active entity as it is created during execution & loaded into main memory.
- It is a dynamic entity
- It has its own control block called Process control block (PCB)
- Exists for limited span of time as it gets terminated after completion of task

▪ Process

- A program in execution is called a process.
- It must progress in sequential fashion.
- It is called as job, task and unit of work.

- Process state / State Transition

(i) As process executes, it changes state:

A] new: Process is being created:

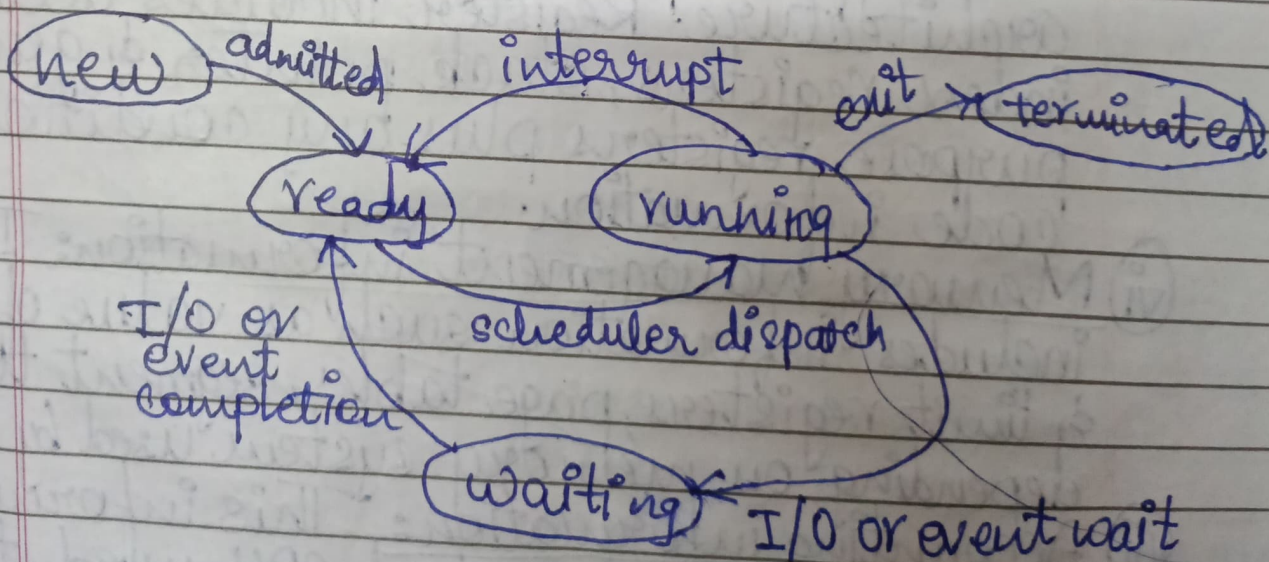
B] ready: Process is waiting to be assigned to a process

C] running: Instructions are being executed

D] waiting: Process is waiting for some event to occur

E] terminated: Process has finished execution

(ii) Diagram:



Process Control Block (PCB)

- i) Each process is represented as a process control block (PCB) in operating system. It contains information associated with specific process.
- ii) Process State: Indicates current state of a process. Process state can be new, ready, running, waiting & terminated.
- iii) Process number: Each process is associated with unique number which is known as process identification number.
- iv) Program counter: Indicates the address of next instruction to be executed for process.
- v) CPU registers: The registers vary in number and type depending on computer architecture. Register includes accumulator, index registers, stack pointer & general purpose registers plus any condition code information.
- vi) Memory management information: It includes information such as value of base & limit registers, page tables, segment tables, depending on memory system used by OS.
- vii) Accounting information: This information includes the amount of CPU used, time limits, account holders, job or process number & so on. It also includes information about listed I/O devices allocated to process such as list of open files.

Each PCB gives information about particular process for which it is designed.

process state
process number
program counter
registers

memory limits
list of open files

...

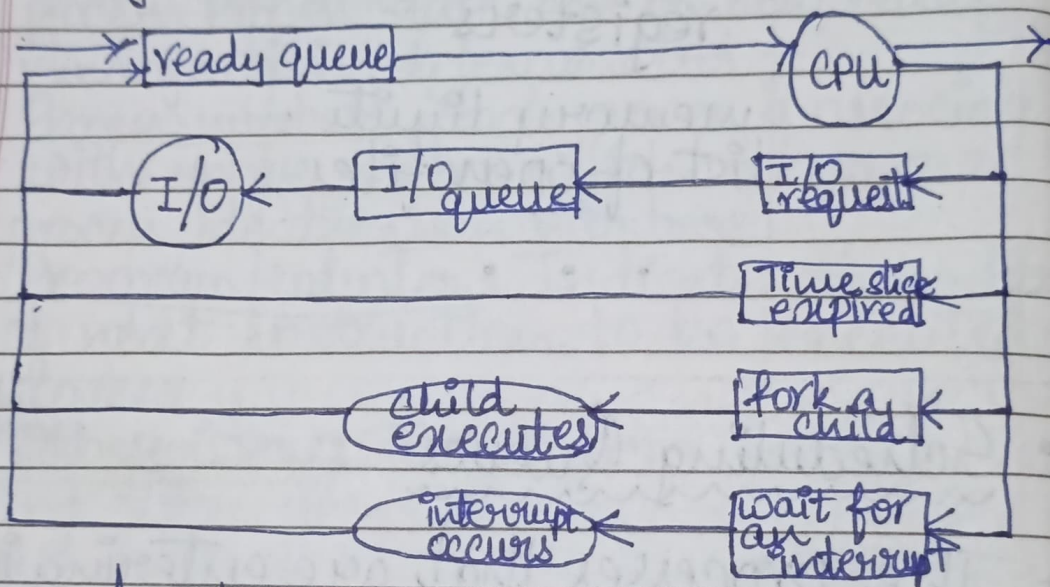
• Scheduling Queues:

- The processes that are entering into the system are stored in Job queue. If processes are in ready state, are generally placed in Ready queue.
- The processes waiting for a device are placed in Device queue. There are unique device queues which are available for every I/O devices.
- First place a new process in Ready queue and then it waits in ready queue till it is selected for execution.
- Once the process is assigned to CPU & is executing, any one of the events occur:
 - ① Process issue an I/O request, and then placed in I/O queue.

(i) Process may create a new sub process and wait for termination.

(ii) Process may be removed forcibly from CPU, which is an interrupt and it is put back in ready queue.

— Diagram:



- In first two cases, the process switches from waiting state to ready state and then puts it back in ready queue.
- A process continues this cycle till it terminates, at which time it is removed from all queues and has its PCB and resources deallocated.

• Types of Schedulers

(i) Long term scheduler:

- It selects programs from job pool and loads them into main memory. It controls degree of multiprogramming.

Schedulers - They are special system software which handle process scheduling in various ways. Their main task is to select the jobs to be submitted into system and to decide which process to run.

- The degree of multiprogramming is the number of processes loaded into main memory. System contains I/O bound processes and CPU bound processes.
- An I/O bound processes spends more time for doing I/O operations whereas CPU bound processes spends more time in doing computations with CPU.
- So it is responsibility of long term scheduler to balance the system by loading some I/O bound and some CPU bound processes in main memory.
- Long term scheduler executes only when a process leaves the system, so it executes less frequently. When long term scheduler selects a process from job pool, the state of process changes from new to ready state.

② Short Term scheduler:

- Also known as CPU scheduler. This scheduler selects processes that are ready for execution from ready queue and allocates CPU to selected processes.
- Frequency of execution of short term scheduler is more than other schedulers.
- When short term scheduler selects a process, that the state of process changes from ready to running state.

③ Medium term scheduler:

- When process is in running state, due to some interrupt it is blocked. System swaps out blocked process and store it into a blocked & swapped out process queue.
- When space is available in main memory, the operating system looks at list of swapped out but ready processes. It selects one process from that list & loads it into ready queue.
- The job of medium term scheduler is to select a process from swapped out process queue & load it into main memory.
- This scheduler works in close conjunction with long term scheduler for loading process into main memory.

▪ Difference: Long Term, Medium Term & Short Term

Short-Term	Medium-Term	Long-Term
① Also called as CPU scheduler	Also called as process swapping scheduler	Also called a job scheduler
② Provides lesser control over degree of multiprogramming	Reduces control over degree of multiprogramming	Controls degree of multiprogramming
③ Speed is very fast	It speed is between short-term & long-term scheduler	Speed is more than medium-term scheduler.

Access ready queue & CPU

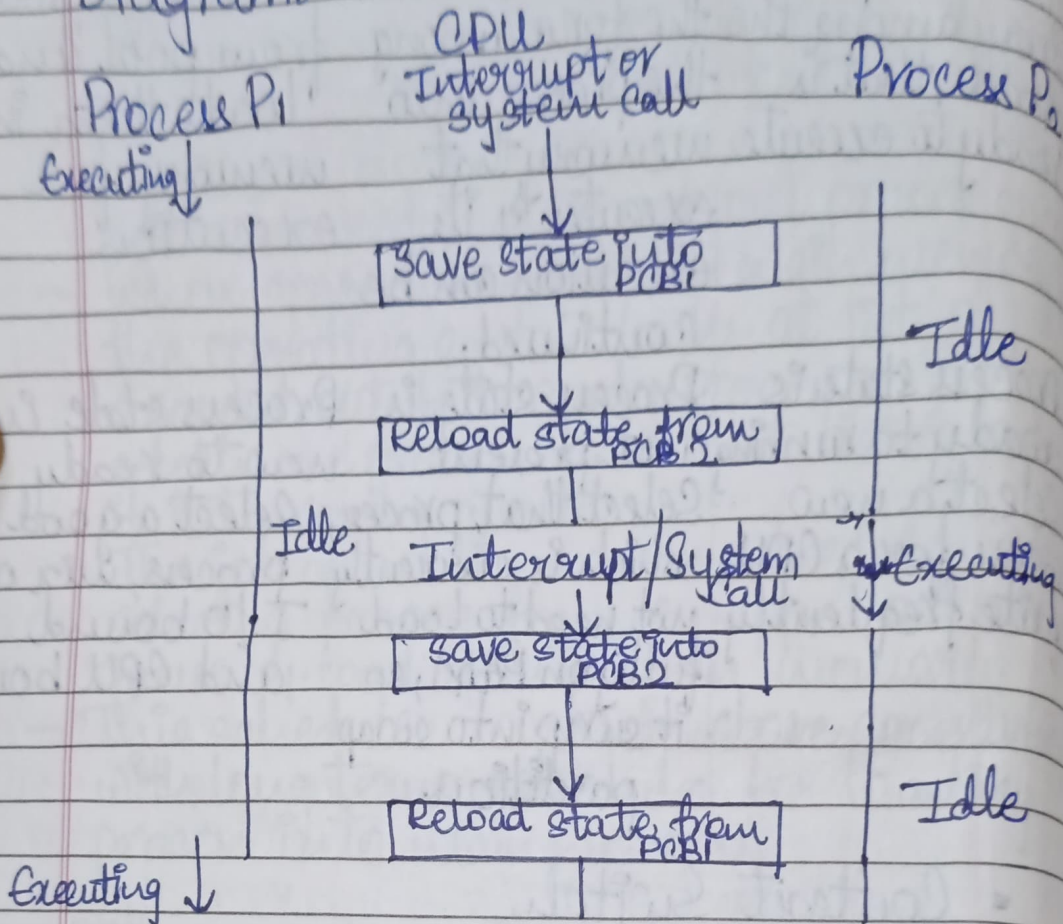
Access job pool & ready queue
Date: ___/___/___

- | | | |
|--|---|---|
| ① It selects processes from among the process that is ready to execute | It can reintroduce the form among the process into memory that executes & its execution can be continued. | It selects processes from pool and loads them into memory for execution |
| ② Process state is ready to running | Process state is not present | Process state is new to ready |
| ③ Select a new process for a CPU quite frequently | Select that process, which is currently not need to load fully on RAM, so it swap into swap partition. | Select a good process, viz of I/O bound and CPU bound |

• Context Switch

- Context switching in an operating system involves saving context or state of a running process so that it can be restored later and then loading the context or state of another process and run it.
- It refers to process/method used by system to change the process from one state to another using CPUs present in system to perform its job
- When process is switched, following information is stored: Program counter, Scheduling information, I/O state, Accounting, Changed state, etc.

- Diagram:



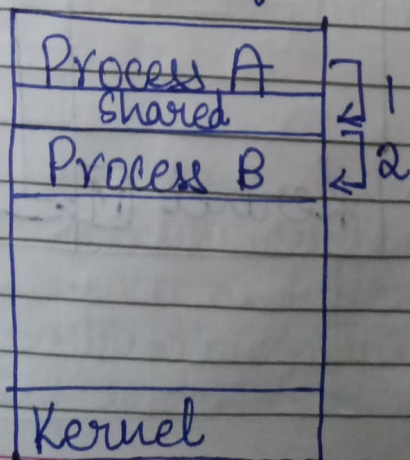
- Inter-process communication (IPC)
- Cooperating processes require an IPC mechanism that will allow them to exchange data & information.
- There are 2 models of IPC: Shared memory, Message passing.

IPC is a mechanism that allows processes to communicate with each other & synchronize their actions.

Date: / /

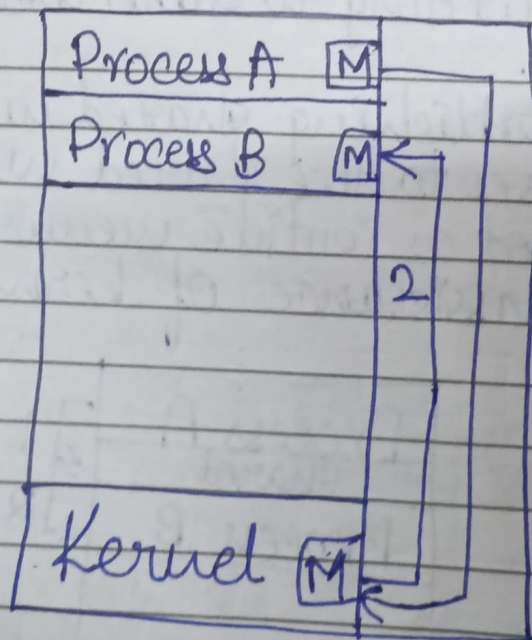
① Shared memory:

- ① In this, all processes who want to communicate with other processes can access a region of memory residing in an address space of a process creating shared memory segment.
- ② All processes using shared memory segment should attach to address space of shared memory. All the processes can exchange information by reading and/or writing data in shared memory segment.
- ③ The form of data & location are determined by these processes who want to communicate with each other.
- ④ These processes are not under control of operating system. These processes are also responsible for ensuring that they are not writing to same location simultaneously.
- ⑤ After establishing shared memory segment, all accesses to the shared memory segment are treated as routine memory access & without assistance of kernel.



② Message Passing:

- ① In this model, communication takes place by exchanging messages between cooperative processes.
- ② It allows processes to communicate & synchronize their action without sharing same address space.
- ③ It is particularly useful in distributed environment where communication processes may reside on different computer connected by a network.
- ④ Communication requires sending & receiving messages through kernel.
- ⑤ The processes that want to communicate with each other must have communication link between them. Between each pair of processes exactly one communication link.



▪ Difference: Shared Memory & Message Passing.

No kernel intervention

Involves kernel intervention

Shared Memory

Message Passing

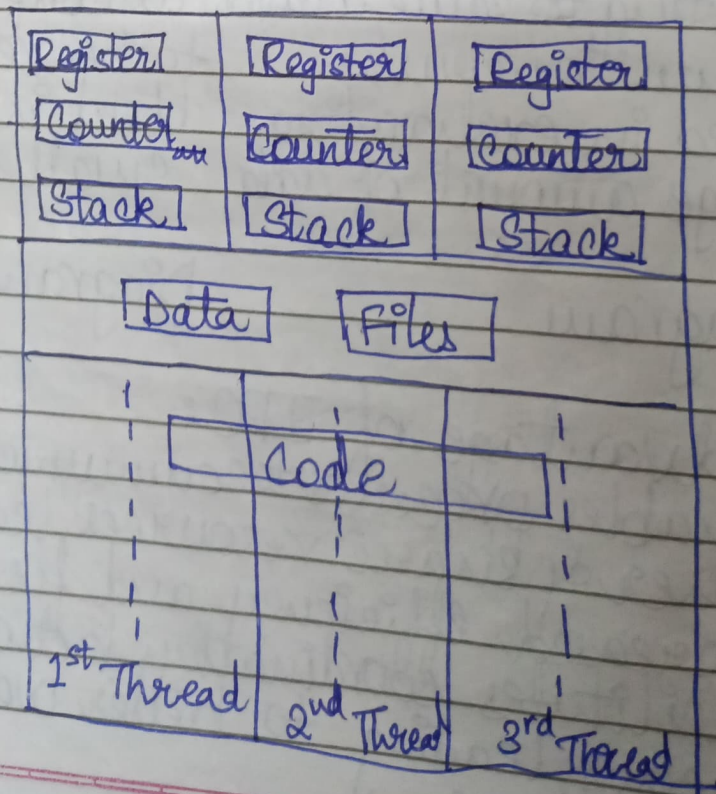
- | | |
|--|--|
| <p>(i) The shared memory region is used for data communication.</p> <p>(ii) Offers maximum speed of computation because communication is completed via shared memory, so system calls are only required to establish shared memory.</p> <p>(iii) Faster communication.</p> <p>(iv) Processes need to ensure that they are not writing to same location simultaneously.</p> <p>(v) Used in exchanging large amount of data.</p> <p>(vi) Diagram</p> | <p>A message passing facility is used for communication.</p> <p>Takes huge time because it is performed via kernel.</p> <p>Slower communication.</p> <p>Useful for sharing small amounts of data as conflicts need not to be resolved.</p> <p>Used in exchanging small amounts of data.</p> <p>Diagram</p> |
|--|--|

— Advantage of IPC:

- (i) Enables processes to communicate with each other & share resources leading to increased efficiency and flexibility.
- (ii) Facilitates coordination between multiple processes, leading to better overall system performance.

• Thread

- A thread is a flow of execution through process code, with its own program counter that keeps track of which instruction to execute next, system registers which hold its current working variables and a stack which contains execution history.
- A thread shares with its peer threads few information like code segment, data segment and open files.
- It is also called a lightweight process.
- They provide a way to improve application performance through parallelism.
- Each thread belongs to exactly one process and no thread can exist outside a process.
- Each thread represents a separate flow of control.



- Advantages:

- (i) Minimize context switching time.
- (ii) Efficient communication
- (iii) Allow utilization of multiprocessor architectures to greater scale & efficiency.

- Types of Thread:

- (1) User Level Threads
- (2) Kernel Level Threads

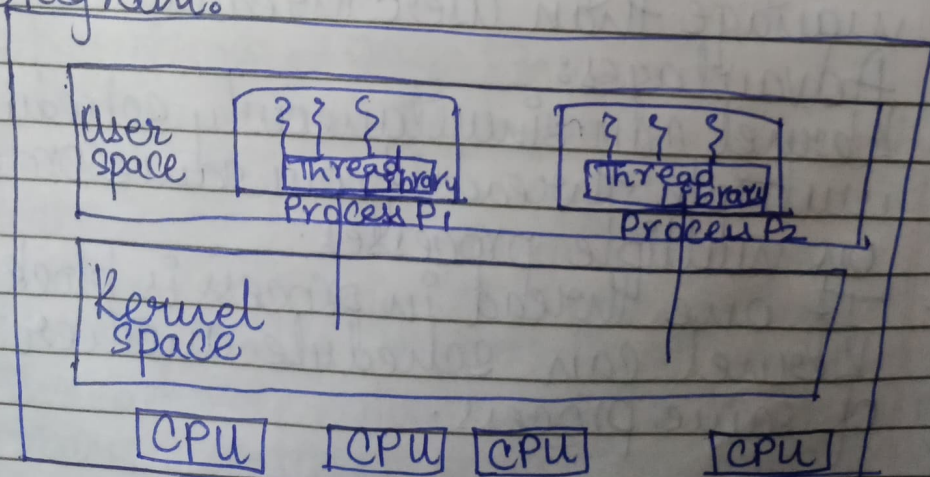
A] User level threads

(i) In this case, thread management kernel is not aware of existence of threads.

(ii) Thread library contains code for creating and destroying threads, for passing message and data between threads, for scheduling thread execution and for saving and restoring thread contexts.

(iii) The application starts with single thread.

(iv) Diagram:



(v) Advantages:

- Thread switching does not require kernel mode privileges.
- User level thread can run on any operating system and are fast to create and manage.

B Kernel level threads

(i) In this case, thread management is done by kernel. There is no thread management code in application area.

(ii) They are supported directly by operating system. Any application can be programmed to be multithreaded.

(iii) All threads within an application are supported within single process.

(iv) Kernel maintains context information for process as a whole & for individual threads within the process.

(v) Scheduling by kernel is done on thread basis. Kernel performs thread creation, scheduling & management in kernel space.

(vi) They are generally slower to create & manage than user threads.

(vii) Advantages:

- Kernel can simultaneously schedule multiple threads from same process on multiple processes.

- If one thread in process is blocked, kernel can schedule another thread of same process.

• Difference between: User level & Kernel level thread

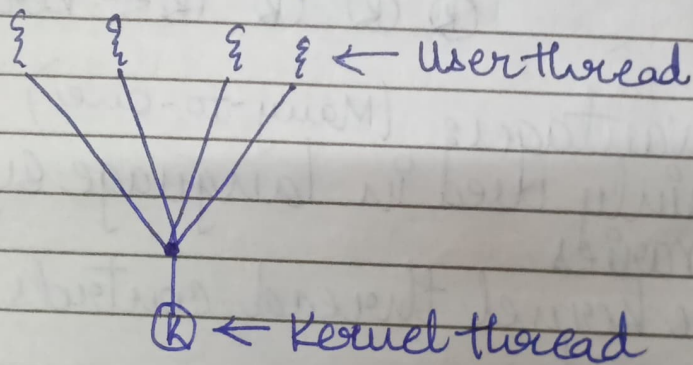
User-level	Kernel-level
i) User threads are implemented by users	Kernel threads are implemented by OS
ii) Implementation is easy	Implementation is complicated
iii) The Operating system doesn't recognize user-level threads	Kernel threads are recognized by Operating system.
iv) Context switch time is less	Context switch time is more
v) Any operating system can support user-level threads.	Kernel-level threads are operating system specific
eg: Java thread, POSIX	eg: Windows Solaris.

■ Multi-threading models:

Many systems provide support for both user and kernel threads, resulting in multithreading models.

① Many-to-One model:

- It wraps many user-level threads to one kernel thread.
- Thread management is done by thread library in user space, so it is efficient; but entire process will block if thread makes blocking system call.
- Also, because only one thread can access kernel at a time, multiple threads are unable to run in parallel on multiprocessors.
- eg: Green threads - A thread library available for Solaris.



- Advantages: (One-to-one)

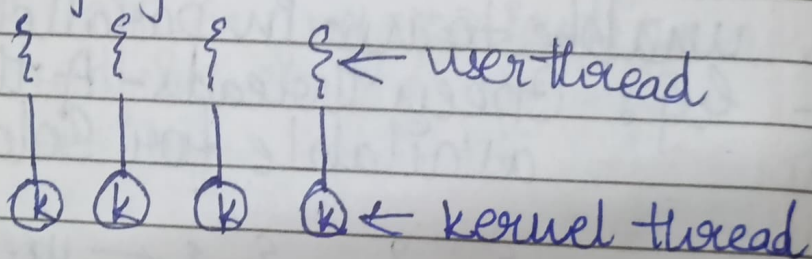
- More concurrency because of multiple threads can run in parallel on multiple CPUs.
- Less complication in processing.

- Disadvantages: (One-to-one)

- Thread creation involves light-weight process creation.
- Kernel thread is an overhead.

⑪ One-to-One model

- It maps each user thread to a kernel thread.
- It provides more concurrency than many-one model by allowing another thread to run when thread makes a blocking system call; it also allows multiple threads to run in parallel on multiprocessors.
- Only drawback to this model is creating user thread required creating corresponding kernel thread.
- eg: Linux, along with family of windows operating systems



- Advantages: (Many-to-one)
 - Mainly used in language system, portable libraries
 - One kernel thread controls multiple user thread.
- Disadvantages: (Many-to-one)
 - Parallelism is not supported.
 - One block can block all user threads.

3 primary thread libraries:

- POSIX Pthreads
- Win32 threads

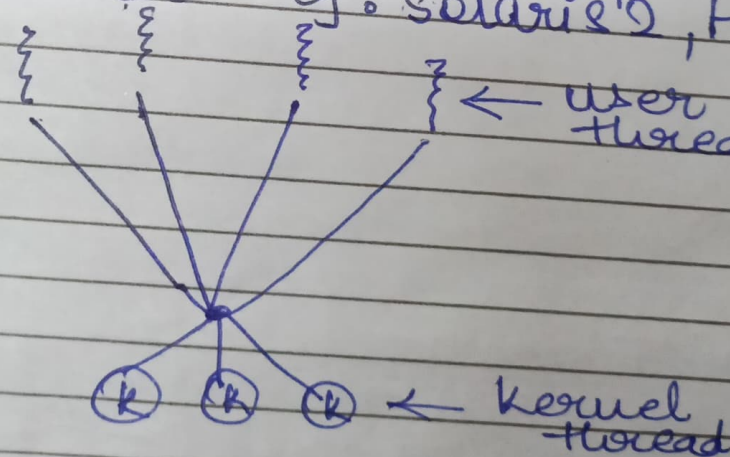
- ~~Unix~~ threads

Date: / /

iii

Many-to-Many Model

- Multiplexes many user-level threads to smaller or equal number of kernel threads.
- Number of kernel threads may be specific to either a particular application or machine.
- The one-to-one model allows for greater concurrency but developer has to be careful not to create too many threads within an application.
- The many-to-many model suffers from neither of these shortcomings. Developers can create as many user threads as necessary & corresponding kernel threads can run in parallel on multiprocessor. eg: Solaris 2, HP-UX, Tru64 UNIX



Advantages:

- Many threads can be created as per user's requirement.
- Multiple kernel or equal to user threads can be created.

Disadvantages:

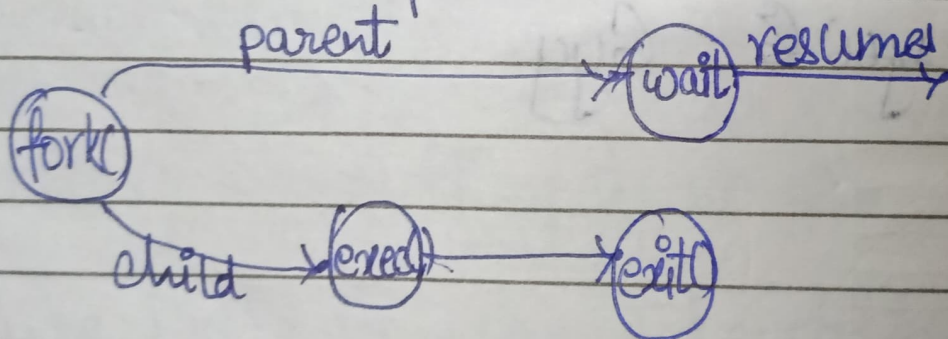
- True concurrency cannot be achieved.

* Process Creation & Termination

i) When new process is to be added to those currently being managed, the operating system builds data structures that are used to manage process & allocates address space in main memory to process.

ii) Parent processes create child processes, which in turn create other processes, forming a tree of processes.

iii) System call `CreateProcess()` in Windows & `fork()` in Unix which tells operating system to create new process.



iv) `exec()` system call after `fork` to replace process memory space with new program.

85

(v) Termination occurs when process completes its task. It invokes system call `ExitProcess()` in Windows and `exit()` in Unix.

(vi) A process may cause abnormal termination of some another process. For this, process invokes system call `TerminateProcess()` in Windows & `kill()` in Unix.